Homework 2

Ulkar Ahmadli

ADA University

Fall 2025 Intro to Big Data Analytics (INFT - 4836 - 10216)

Umid Suleymanov

Task 1: Spark RDDs – Core Transformations & Actions

Dataset Selection

I have selected Comprehensive Diabetes Clinical Dataset from Kaggle and downloaded it.

The link: https://www.kaggle.com/datasets/priyamchoksi/100000-diabetes-clinical-dataset

This dataset contains health data of 100,000 individuals that includes 16 columns about gender, age, location, race, hypertension, heart disease, smoking history, BMI, and more.

Setup

I started with downloading Apache Spark to my device through homebrew package manager. Spark version 4.0.1 downloaded with all required dependencies as OpenJDK 21 and more.

```
nijatahmadli@ulkarahmadli ~ % brew install apache-spark

✔ JSON API cask.jws.json                        [Downloaded   15.0MB/ 15.0MB]
✔ JSON API formula.jws.json                     [Downloaded   31.7MB/ 31.7MB]
==> Fetching downloads for: apache-spark
✔ Bottle Manifest apache-spark (4.0.1_1)        [Downloaded   10.1KB/ 10.1KB]
✔ Bottle Manifest gettext (0.26_1)              [Downloaded   13.8KB/ 13.8KB]
✔ Bottle Manifest icu4c@78 (78.1)               [Downloaded    9.7KB/  9.7KB]
✔ Bottle Manifest openjdk@21 (21.0.9)           [Downloaded   39.7KB/ 39.7KB]
✔ Bottle Manifest libpng (1.6.51)               [Downloaded    8.5KB/  8.5KB]
✔ Bottle Manifest pcre2 (10.47)                 [Downloaded   10.4KB/ 10.4KB]
✔ Bottle Manifest glib (2.86.2)                 [Downloaded   20.3KB/ 20.3KB]
✔ Bottle Manifest harfbuzz (12.2.0_1)           [Downloaded   29.9KB/ 29.9KB]
✔ Bottle libpng (1.6.51)                        [Downloaded  453.5KB/453.5KB]
✔ Bottle gettext (0.26_1)                       [Downloaded    9.6MB/  9.6MB]
✔ Bottle icu4c@78 (78.1)                        [Downloaded   31.8MB/ 31.8MB]
✔ Bottle pcre2 (10.47)                          [Downloaded    2.4MB/  2.4MB]
✔ Bottle harfbuzz (12.2.0_1)                    [Downloaded    3.5MB/  3.5MB]
✔ Bottle glib (2.86.2)                          [Downloaded    8.8MB/  8.8MB]
✔ Bottle openjdk@21 (21.0.9)                    [Downloaded  202.0MB/202.0MB]
✔ Bottle apache-spark (4.0.1_1)                 [Downloaded  549.4MB/549.4MB]
==> Installing dependencies for apache-spark: gettext, icu4c@78 and openjdk@21
==> Installing apache-spark dependency: gettext
==> Pouring gettext--0.26_1.sonoma.bottle.tar.gz
🍺  /usr/local/Cellar/gettext/0.26_1: 2,428 files, 28.3MB
==> Installing apache-spark dependency: icu4c@78
==> Pouring icu4c@78--78.1.sonoma.bottle.tar.gz
🍺  /usr/local/Cellar/icu4c@78/78.1: 279 files, 86.7MB
==> Installing apache-spark dependency: openjdk@21
==> Pouring openjdk@21--21.0.9.sonoma.bottle.tar.gz
🍺  /usr/local/Cellar/openjdk@21/21.0.9: 600 files, 346.4MB
==> Installing apache-spark
==> Pouring apache-spark--4.0.1_1.all.bottle.tar.gz
🍺  /usr/local/Cellar/apache-spark/4.0.1_1: 2,343 files, 604.6MB
==> Running `brew cleanup apache-spark`...
Disable this behaviour by setting `HOMEBREW_NO_INSTALL_CLEANUP=1`.
Hide these hints with `HOMEBREW_NO_ENV_HINTS=1` (see `man brew`).
```

I created a Python virtual environment using python3 -m venv venv and activated it for keeping my Spark installation separated from system packages.

```
● nijatahmadli@ulkarahmadli ~ % python3 -m venv venv

● nijatahmadli@ulkarahmadli ~ % source venv/bin/activate
```

Then I installed PySpark library inside the virtual environment.

```
(base) nijatahmadli@ulkarahmadli ~ % pip3 install pyspark
Collecting pyspark
  Downloading pyspark-4.0.1.tar.gz (434.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 434.2/434.2 MB 6.4 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Collecting py4j==0.10.9.9 (from pyspark)
  Downloading py4j-0.10.9.9-py2.py3-none-any.whl.metadata (1.3 kB)
Downloading py4j-0.10.9.9-py2.py3-none-any.whl (203 kB)
Building wheels for collected packages: pyspark
  DEPRECATION: Building 'pyspark' using the legacy setup.py bdist_wheel mechanism, which will
be removed in a future version. pip 25.3 will enforce this behaviour change. A possible replac
ement is to use the standardized build interface by setting the `--use-pep517` option, (possib
ly combined with `--no-build-isolation`), or adding a `pyproject.toml` file to the source tree
 of 'pyspark'. Discussion can be found at https://github.com/pypa/pip/issues/6334
  Building wheel for pyspark (setup.py) ... done
  Created wheel for pyspark: filename=pyspark-4.0.1-py2.py3-none-any.whl size=434813798 sha256
=6342db219f1f3b72a80f513593e7bd5415fd87899ed2dbb91d43211ca4df325d
  Stored in directory: /Users/ulkarahmadli/Library/Caches/pip/wheels/00/e3/92/8594f4cee2c9fd4a
d82fe85e4bf2559ab8ea84ef19b1dd3d15
Successfully built pyspark
Installing collected packages: py4j, pyspark
Successfully installed py4j-0.10.9.9 pyspark-4.0.1
```

After successfully installed, I checked the setup to confirm.

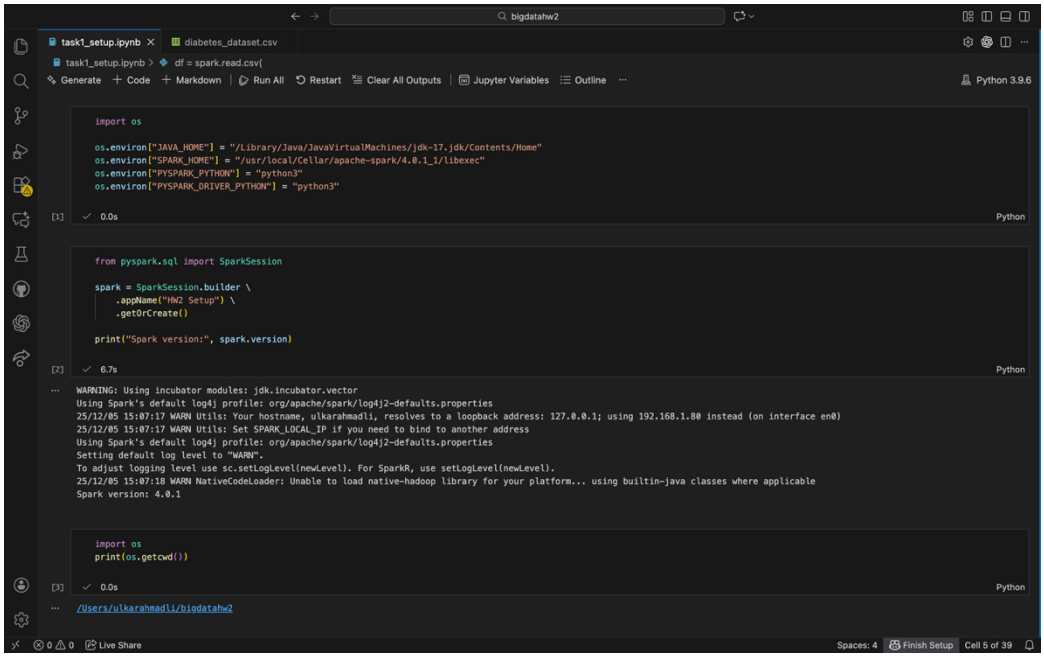
```
nijatahmadli@ulkarahmadli ~ % pyspark

Python 3.9.6 (default, Apr 30 2025, 02:07:18)
[Clang 17.0.0 (clang-1700.0.13.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
WARNING: Using incubator modules: jdk.incubator.vector
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
25/12/03 20:55:52 WARN Utils: Your hostname, ulkarahmadli, resolves to a loopback address: 127.0.0.1; using 192.168.1.80 inst
ead (on interface en0)
25/12/03 20:55:52 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
25/12/03 20:55:53 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes
 where applicable
25/12/03 20:55:54 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /__ / .__/\_,_/_/ /_/\_\   version 4.0.1
      /_/

Using Python version 3.9.6 (default, Apr 30 2025 02:07:18)
Spark context Web UI available at http://192.168.1.80:4041
Spark context available as 'sc' (master = local[*], app id = local-1764780954141).
SparkSession available as 'spark'.
>>>
```

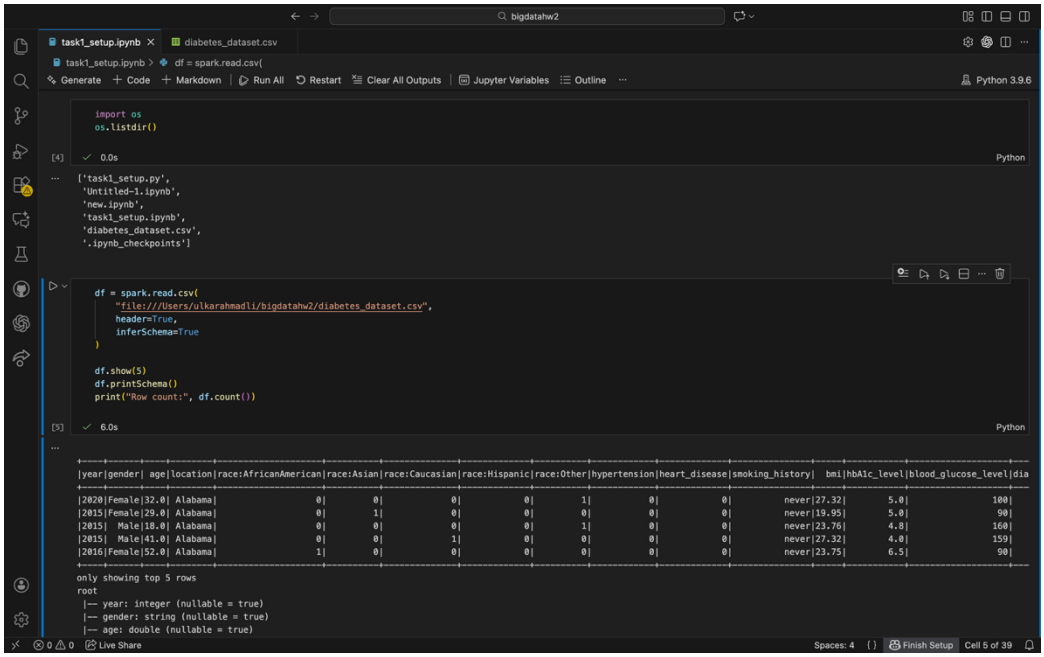
To be able to run Spark code and capture clean outputs, I installed Jupyter Notebook. This helped me to execute Spark code inside a .ipynb notebook.

```
(venv) nijatahmadli@ulkarahmadli ~ % pip install jupyter

  Collecting jupyter
    Downloading jupyter-1.1.1-py2.py3-none-any.whl.metadata (2.0 kB)
  Collecting notebook (from jupyter)
    Downloading notebook-7.5.0-py3-none-any.whl.metadata (10 kB)
  Collecting jupyter-console (from jupyter)
    Downloading jupyter_console-6.6.3-py3-none-any.whl.metadata (5.8 kB)
  Collecting nbconvert (from jupyter)
    Downloading nbconvert-7.16.6-py3-none-any.whl.metadata (8.5 kB)
  Collecting ipykernel (from jupyter)
    Downloading ipykernel-6.31.0-py3-none-any.whl.metadata (4.5 kB)
  Collecting ipywidgets (from jupyter)
    Downloading ipywidgets-8.1.8-py3-none-any.whl.metadata (2.4 kB)
  Collecting jupyterlab (from jupyter)
    Downloading jupyterlab-4.5.0-py3-none-any.whl.metadata (16 kB)
  Collecting appnope>=0.1.2 (from ipykernel->jupyter)
    Downloading appnope-0.1.4-py2.py3-none-any.whl.metadata (908 bytes)
  Collecting comm>=0.1.1 (from ipykernel->jupyter)
⊗ 0 ⚠ 1  Live Share   Ln 16, Col 11   Spaces: 4   UTF-8   LF   { } Python   Finish S
```

Data Loading

First, I configured the environment variables for Java version and Spark installation path for PySpark. Second, I created a SparkSession using the standard builder. Then before loading my dataset, I checked my working directory using os.getcwd(), and available files with os.listdir().





Finally, I loaded my dataset into a Spark DataFrame. I used header = True and inferSchema = True to detect column names and data types automatically. I displayed first 5 rows.

RDD Transformations & Actions

1. **map() transformation**

```
rdd_map = rdd.map(lambda row: (row.year, row.age, row.diabetes))
rdd_map.take(5)
```

[7] ✓ 0.5s

```
[(2020, 32.0, 0),
 (2015, 29.0, 0),
 (2015, 18.0, 0),
 (2015, 41.0, 0),
 (2016, 52.0, 0)]
```

I used map for creating a new RDD which contains only selected fields for each row. In my case, I extracted year, age, and diabetes indicator for each record. It helped me to focus on the fields that are only needed for analysis.

2. **filter() transformation**

```
rdd_filtered = rdd.filter(lambda row: row.age is not None and row.age >= 18)
rdd_filtered.take(5)
```

[8] ✓ 0.5s Python

```
[Row(year=2020, gender='Female', age=32.0, location='Alabama', race:AfricanAmerican=0, race:Asian=0, race:Caucasian=0, race:Hispanic=0, race:Other=1, hypertension=0, heart_disease=0,
 Row(year=2015, gender='Female', age=29.0, location='Alabama', race:AfricanAmerican=0, race:Asian=1, race:Caucasian=0, race:Hispanic=0, race:Other=0, hypertension=0, heart_disease=0,
 Row(year=2015, gender='Male', age=18.0, location='Alabama', race:AfricanAmerican=0, race:Asian=0, race:Caucasian=0, race:Hispanic=0, race:Other=1, hypertension=0, heart_disease=0, sm
 Row(year=2015, gender='Male', age=41.0, location='Alabama', race:AfricanAmerican=0, race:Asian=0, race:Caucasian=1, race:Hispanic=0, race:Other=0, hypertension=0, heart_disease=0, sm
 Row(year=2016, gender='Female', age=52.0, location='Alabama', race:AfricanAmerican=1, race:Asian=0, race:Caucasian=0, race:Hispanic=0, race:Other=0, hypertension=0, heart_disease=0,
```

I used filter transformation for removing rows that don't meet a specific condition. I only kept records where age is not null and the individual is 18 or older. This helped me to focus on adults in the dataset.

3. **flatMap() transformation**

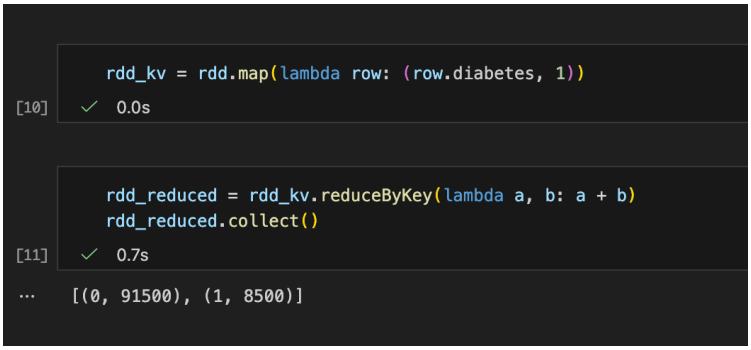
```
rdd_flat = rdd.flatMap(lambda row: row.smoking_history.split(" "))
rdd_flat.take(10)
```

[9] ✓ 0.4s

```
['never',
 'never',
 'never',
 'never',
 'never',
 'not',
 'current',
 'current',
 'No',
 'Info']
```

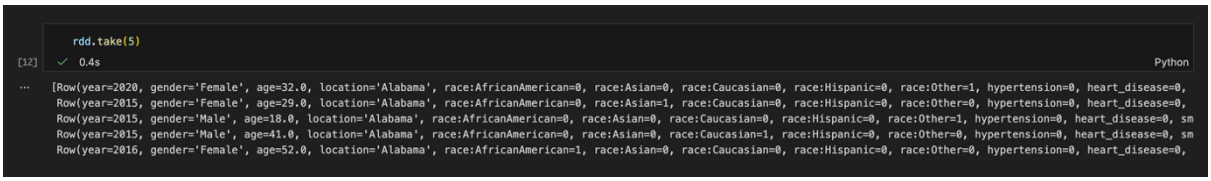
Here, I used flatmap for splitting the smoking_history field into individual tokens. As there are some entries that contain multiple words, this transformation helped me to return multiple outputs for a single input row and then flatten them into a single list.

4. **reduceByKey()**



First, I converted the dataset into key-value pairs where key is diabetes label (0 or 1) and the value is 1. Then using this operation, I summed the values for each key. This gave me the total number of people with and without diabetes.

5. **take()**



This operation retrieves the first n elements from an RDD. I used take(5) throughout the analysis to see the content of new RDDs. It confirms that the transformation applied correctly without printing the whole dataset. Since take is an action, it triggers the execution of the transformations and returns accrual results.

Task 2: Spark SQL – Querying Distributed Data

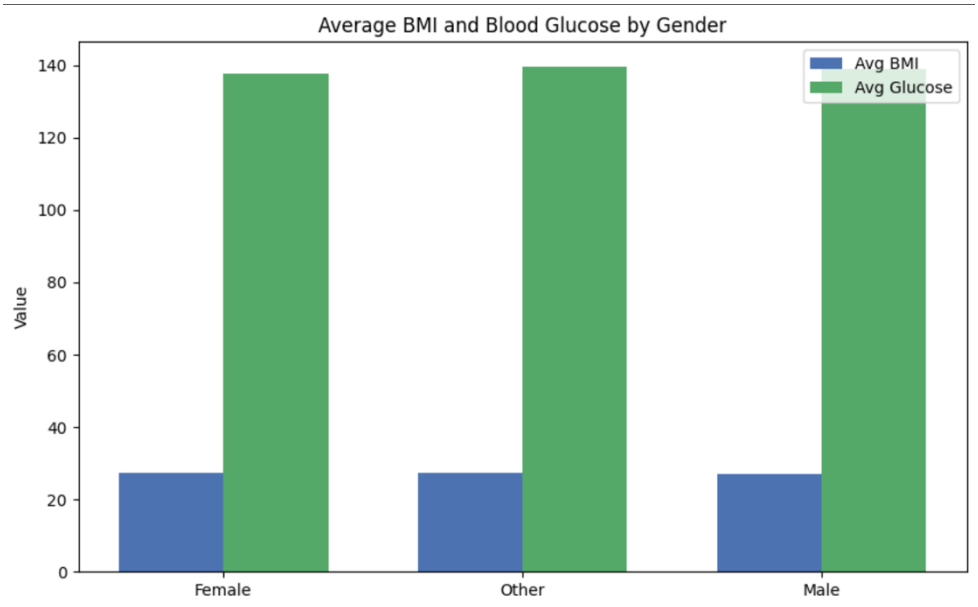
1. Aggregation Query

```
result_agg = spark.sql("""
    SELECT
        gender,
        COUNT(*) AS total_patients,
        ROUND(AVG(bmi), 2) AS avg_bmi,
        ROUND(AVG(blood_glucose_level), 2) AS avg_glucose
    FROM diabetes
    GROUP BY gender
""")

result_agg.show()
```

[16] ✓ 1.1s

```
+------+--------------+-------+-----------+
|gender|total_patients|avg_bmi|avg_glucose|
+------+--------------+-------+-----------+
|Female|         58552|  27.45|     137.47|
| Other|            18|  27.38|     139.44|
|  Male|         41430|  27.14|     138.89|
+------+--------------+-------+-----------+
```

I used Spark SQL for computing aggregated statistics grouped by gender. This query calculates the number of patients in each gender category and computes the average BMI, glucose level for each group. It is good for identifying whether physiological characteristics differ across gender groups.



2. **Filtering Query**

```
result_filter = spark.sql("""
    SELECT
        year, gender, age, blood_glucose_level, hypertension
    FROM diabetes
    WHERE age > 50
      AND hypertension = 1
      AND blood_glucose_level > 150
""")

result_filter.show(10)
```

[20] ✓ 0.1s

```
+----+------+----+-------------------+------------+
|year|gender| age|blood_glucose_level|hypertension|
+----+------+----+-------------------+------------+
|2015|Female|66.0|                200|           1|
|2016|Female|80.0|                159|           1|
|2016|  Male|80.0|                200|           1|
|2015|Female|72.0|                160|           1|
|2015|  Male|57.0|                240|           1|
|2015|Female|67.0|                160|           1|
|2015|Female|59.0|                155|           1|
|2019|  Male|74.0|                240|           1|
|2019|  Male|52.0|                280|           1|
|2019|Female|64.0|                155|           1|
+----+------+----+-------------------+------------+
only showing top 10 rows
```

I used WHERE in SQL with different logical conditions to filter the dataset. The query takes patients older than 50 who also have hypertension and elevated blood glucose levels above 150. This filtering helped to identify a specific high-risk subgroup within the dataset.

3. **Join Query**

I started with creating a small lookup table that assigns a full descriptive label to each gender category to be able to demonstrate SQL joins in Spark. Then, I performed a left join between the main dataset and this lookup table using the gender column as the join key. It is really helpful because it enriches the dataset with additional descriptive information. Also, it makes the results more readable and allowing future analyses to use more understandable labels rather than coded values.

```
from pyspark.sql import Row

gender_lookup = spark.createDataFrame([
    Row(gender="Male", gender_full="Male Patient"),
    Row(gender="Female", gender_full="Female Patient")
])

gender_lookup.createOrReplaceTempView("gender_lookup")
```

✓ 0.0s

```
result_join = spark.sql("""
    SELECT
        d.gender,
        g.gender_full,
        d.age,
        d.bmi,
        d.diabetes
    FROM diabetes d
    LEFT JOIN gender_lookup g
        ON d.gender = g.gender
""")

result_join.show(10)
```

✓ 0.4s

```
+------+-------------+----+-----+--------+
|gender|  gender_full| age|  bmi|diabetes|
+------+-------------+----+-----+--------+
|Female|Female Patient|32.0|27.32|       0|
|Female|Female Patient|29.0|19.95|       0|
|Female|Female Patient|52.0|23.75|       0|
|Female|Female Patient|49.0|24.34|       0|
|Female|Female Patient|15.0|20.98|       0|
|  Male|  Male Patient|18.0|23.76|       0|
|  Male|  Male Patient|41.0|27.32|       0|
|  Male|  Male Patient|66.0|27.32|       0|
|  Male|  Male Patient|51.0|38.14|       0|
|  Male|  Male Patient|42.0|27.32|       0|
+------+-------------+----+-----+--------+
only showing top 10 rows
```

Task 3: Spark DataFrames – Data Cleaning & Transformations

1. Data Cleaning: Handling Missing or Incorrect Values

```
from pyspark.sql.functions import col, sum as spark_sum

null_counts = df.select([spark_sum(col(c).isNull().cast("int")).alias(c) for c in df.columns])
null_counts.show()
```

✓ 0.7s Python

```
+----+------+---+--------+-----------------+---------+-----------+-----------+--------+------------+-------------+---------------+---+---------+-------------------+------+
|year|gender|age|location|race:AfricanAmerican|race:Asian|race:Caucasian|race:Hispanic|race:Other|hypertension|heart_disease|smoking_history|bmi|hbA1c_level|blood_glucose_level|diabet
+----+------+---+--------+-----------------+---------+-----------+-----------+--------+------------+-------------+---------------+---+---------+-------------------+------+
|   0|     0|  0|       0|                0|        0|          0|          0|       0|           0|            0|              0|  0|        0|                  0|     0|
+----+------+---+--------+-----------------+---------+-----------+-----------+--------+------------+-------------+---------------+---+---------+-------------------+------+
```

First, I checked if there are any missing values written as null over each column. The results showed that all columns have zero null entries. Even though there weren't any missing values, I did the required cleaning techniques.

```
df_dropna = df.dropna()
df_dropna.count(), df.count()
```

24] ✓ 0.4s

```
(100000, 100000)
```

I applied dropna() to show how Spark removes rows that contains missing data. Since, I had no nulls that's why the row count remained same.

```
from pyspark.sql.functions import avg

mean_bmi = df.select(avg("bmi")).first()[0]
mean_age = df.select(avg("age")).first()[0]

df_filled = df.fillna({
    "bmi": mean_bmi,
    "age": mean_age
})

df_filled.show(5)
```

5] ✓ 0.7s Python

```
+----+------+----+--------+--------------------+----------+--------------+-------------+----------+------------+-------------+---------------+-----+-----------+-------------------+---
|year|gender| age|location|race:AfricanAmerican|race:Asian|race:Caucasian|race:Hispanic|race:Other|hypertension|heart_disease|smoking_history|  bmi|hbA1c_level|blood_glucose_level|dia
+----+------+----+--------+--------------------+----------+--------------+-------------+----------+------------+-------------+---------------+-----+-----------+-------------------+---
|2020|Female|32.0| Alabama|                   0|         0|             0|            0|         1|           0|            0|          never|27.32|        5.0|                100|
|2015|Female|29.0| Alabama|                   0|         1|             0|            0|         0|           0|            0|          never|19.95|        5.0|                 90|
|2015|  Male|18.0| Alabama|                   0|         0|             0|            0|         1|           0|            0|          never|23.76|        4.8|                160|
|2015|  Male|41.0| Alabama|                   0|         0|             1|            0|         0|           0|            0|          never|27.32|        4.0|                159|
|2016|Female|52.0| Alabama|                   1|         0|             0|            0|         0|           0|            0|          never|23.75|        6.5|                 90|
+----+------+----+--------+--------------------+----------+--------------+-------------+----------+------------+-------------+---------------+-----+-----------+-------------------+---
only showing top 5 rows
```

Then, I did fillna() by computing the mean of numeric columns which are bmi and age, and replacing potential nulls with these average values. This method is helpful when we want to keep all records without any data loss.

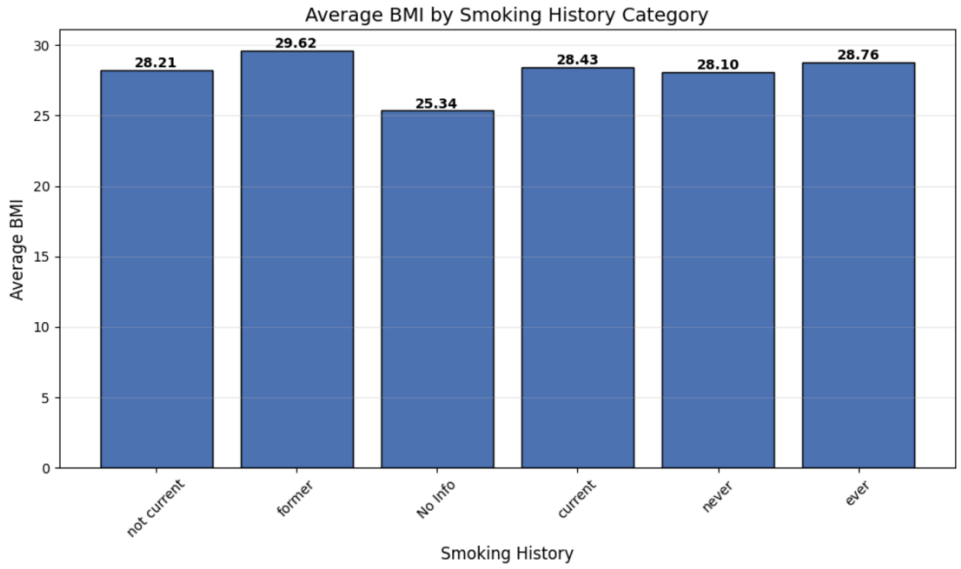
2. groupBy() + agg() operations

```
from pyspark.sql.functions import avg, stddev, count

df_group = df.groupBy("smoking_history").agg(
    count("*").alias("total_patients"),
    avg("bmi").alias("avg_bmi"),
    stddev("bmi").alias("stddev_bmi")
)

df_group.show()
```

[26] ✓ 0.4s

```
+---------------+--------------+------------------+------------------+
|smoking_history|total_patients|           avg_bmi|        stddev_bmi|
+---------------+--------------+------------------+------------------+
|    not current|          6447|28.205087637661492|6.230714928550599|
|         former|          9352|29.620746364414675|6.378794499718356|
|        No Info|         35816|25.343505416573205|6.387415052576948|
|        current|          9286|28.432967908680602|6.224652677781554|
|          never|         35095|28.104620886165623|6.658454420178741|
|           ever|          4004|28.761760739260833|6.344743048259379|
+---------------+--------------+------------------+------------------+
```

Here, I grouped the dataset by the categorical field smoking_history. For each group, I computed number of patients, average and standard deviation of BMI. Grouping by smoking history helps to compare health patterns across different lifestyle groups.



3. orderBy() / sort() with multiple columns

```
df_sorted = df.orderBy(
    col("diabetes").desc(),
    col("age").desc()
)

df_sorted.show(10)
```

[27] ✓ 0.3s Python

```
+----+------+----+--------------+--------------------+----------+--------------+----------+------------+------------+---------------+-----+-----------+-----------------+
|year|gender| age|      location|race:AfricanAmerican|race:Asian|race:Caucasian|race:Hispanic|race:Other|hypertension|heart_disease|smoking_history|  bmi|hbA1c_level|blood_glucose_lev|
+----+------+----+--------------+--------------------+----------+--------------+----------+----------+------------+-------------+---------------+-----+-----------+-----------------+
|2019|Female|80.0|North Carolina|                   0|         0|             0|         1|         0|           0|            0|          never|18.36|        8.8|                2|
|2019|  Male|80.0|       Alabama|                   0|         1|             0|         0|         0|           1|            1|         former|22.58|        6.1|                2|
|2019|Female|80.0|North Carolina|                   0|         0|             1|         0|         0|           0|            0|         former| 29.6|        5.8|                2|
|2019|Female|80.0|       Alabama|                   0|         0|             0|         1|         0|           1|            0|          never|42.71|        5.7|                1|
|2019|Female|80.0|North Carolina|                   0|         0|             0|         1|         0|           0|            0|       No Info|22.74|        7.5|                1|
|2019|  Male|80.0|       Alabama|                   0|         0|             0|         0|         1|           0|            0|         former|27.32|        7.5|                2|
|2019|Female|80.0|North Carolina|                   0|         0|             0|         0|         1|           0|            0|           ever|31.39|        8.8|                2|
|2016|Female|80.0|       Alabama|                   0|         0|             0|         1|         0|           0|            1|          never| 26.0|        7.5|                2|
|2015|  Male|80.0|  North Dakota|                   0|         0|             0|         0|         1|           0|            1|         former|24.36|        7.5|                2|
|2019|  Male|80.0|       Alabama|                   0|         0|             0|         1|         0|           1|            0|       No Info|27.32|        7.5|                1|
+----+------+----+--------------+--------------------+----------+--------------+----------+----------+------------+-------------+---------------+-----+-----------+-----------------+
only showing top 10 rows
```

I sorted the dataset by diabetes indicator in descending order so that diabetic individuals appear at the top, and then by age in descending order to show the oldest individuals within each diabetes category first for showing multiple column sorting. Spark did this action by sorting separate partitions in parallel and then merging them into one ordered result.